Explicit Strategy Verification

Recognising and repairing invalid reasoning strategies under structural perturbations: a gold-free verification method that beats majority voting

94.1% ESV vs **88.8%** CoT maj@4 **89.7%** USC

GSM8K held-out · 813 instances · 3 solvers · +5.3 pp, p = 1.6e-11

By: Ekansh Agrawal and Ritvij Sharma

Main Idea

When a problem is perturbed so its familiar strategy no longer applies, models reuse it anyway. Can a model notice this from the problem text alone, with no answer key — and does that beat trusting the majority vote?

RQ.1 Can a verifier detect an invalid strategy from the problem's own givens?

RQ.2 Does that signal beat the free majority vote it must overturn?

RQ.3 Does it matter who verifies — the solver itself, or an independent model?

RQ.4 Does ESV beat Direct QA, CoT maj@4 and Universal Self-Consistency?

Every method starts from the same four CoT samples; gold is used only to score, so a method can damage the baseline as easily as improve it.

The four methods compared

method	extra calls	evidence it uses
Direct QA maj@4	0	none (no reasoning)
CoT maj@4	0	the vote
Universal Self-Consistency	1	the vote, read in prose
ESV (ours)	1	the problem's givens

All four pick among answers the solver has already produced; none re-solves the problem. The question is which evidence picks best.

ESV is the only one that cannot see how many samples reached each answer.

198 frozen GSM8K pairs (original + perturbed) x 3 open-weight solvers x 4 samples per method.

Literature Review + Gaps (Perturbation Robustness)

JEEBench (Arora et al., 2023): https://arxiv.org/abs/2305.15074

Summary: A benchmark of challenging JEE Advanced problems in physics, chemistry and mathematics. Even the strongest models score far below top human candidates, failing at long-horizon algebra and concept retrieval.

Gap: Scores only the final answer, and its failures are dominated by raw capability. We ran ESV here first and measured why that makes it the wrong testbed: only 5-10% of wrong answers come from reusing the original strategy.

MATH-Perturb (Huang et al., 2025): https://arxiv.org/abs/2502.06453

Summary: Applies simple and hard perturbations to competition problems. Hard perturbations invalidate the original solution and accuracy drops sharply: models blindly reuse familiar strategies.

Gap: Diagnoses the failure mode but proposes no remedy, and never scores whether a model can detect the broken strategy. Our benchmark pairs every problem with a verified strategy-breaking twin and scores the detection.

Literature Review + Gaps (Self-Correction and Voting)

LLMs Cannot Self-Correct Reasoning Yet (Huang et al., 2024): https://arxiv.org/abs/2310.01798

Summary: Intrinsic self-correction — asking a model to review its own answer without external feedback — does not reliably improve reasoning and can make it worse; earlier reported gains relied on oracle labels.

Gap: Tests generic review prompting. We confirm the result and explain it structurally: our self-verification arm loses 3.7 pp, and we measure why: its verdict is far too weak to overturn the majority vote it contradicts.

Universal Self-Consistency (Chen et al., 2023): https://arxiv.org/abs/2311.17311

Summary: Replaces exact-match majority voting with a single call that shows the model all its samples and asks it to select the most consistent response. Our primary baseline.

Gap: It is still a vote: shown the duplicates and asked for consensus, it barely improves on majority voting (67.3% vs 64.1% on our disagreeing instances; +0.9 pp overall, 14 fixes / 7 breaks, $p = 0.189$). ESV hides multiplicity and verifies against the text instead.

What we add: a gold-free trigger, verification anchored in quoted givens rather than generic review, and the finding that verifier independence — not the protocol alone — decides whether verification can beat voting.

Building the Benchmark

1

Step 1: Generate

Model A (DeepSeek V4 Flash) reads P and its answer, names the strategy S that solves it, and changes one structural condition so S breaks. It outputs P-prime, the changed condition, why S now fails, and an alternative strategy S-prime.

2

Step 2: Verify blind

Models B (Qwen 3.5 35B) and C (GPT-OSS-120B) solve P-prime without seeing P or A's answer, then apply the original strategy S to P-prime and label it STILL_VALID or INVALID. The generator never grades its own work.

3

Step 3: Validate and freeze

B and C must agree on the answer with confidence >= 0.6 and agree on the strategy label. Accepted pairs are frozen and shown identically to every method and model. Up to 3 attempts, with verifier feedback on rejection.

Final dataset

198

GSM8K pairs accepted and frozen (99 breaking / 99 preserving)

182 accepted on the first attempt; 2 problems rejected after 3 tries

The trap answer equals the original gold in 94 of 99 breaking pairs

19 pairs re-solved by hand: all correct. 6 GSM8K originals with erroneous published labels are excluded from every method symmetrically.

GSM8K was chosen over JEEBench because its base accuracy (~90%) makes perturbation failures strategy failures rather than capability failures.

What Is a Strategy-Breaking Perturbation?

A perturbation changes one structure, relationship or constraint so that the strategy which solved the original no longer applies, while the new problem stays well-defined, solvable and superficially similar.

Original P (GSM8K test, pair 131)

"In 7 days, Sofie will walk twice as far as Brian. If Sofie plans to walk 10 miles every day, how many miles will Brian walk in seven days?"

Strategy S: 7 x 10 = 70, then halve it — two 7-day totals are compared.

Answer: 35

Perturbed P-prime

"... twice as far as **Brian walks in one day**. Sofie plans to walk 10 miles every day, and Brian walks the same distance every day."

What breaks: the comparison is now a 7-day total vs a one-day distance, so 7(10) = 2b gives Brian's DAILY distance.

Reusing S still answers **35** · correct answer **245**

Why this is the designed stress case: three words change and the trap answer is exactly the original problem's gold. Gemma 3 4B produced 35 in 3 of 4 samples — and each of those samples closed by verifying itself: "Indeed, 70 is twice 35, so our solution is correct." The check confirms the misreading.

The ESV v3 Pipeline (gold-free)

No stage ever sees the gold answer. ESV fires on disagreement among the solver's own samples, so it can damage a correct majority as easily as repair a wrong one — and is scored that way.

1 Sampling

Each solver answers 8 times: 4 Direct QA and 4 CoT at T = 0.7. Tag-structured output, normalized answers. Shared by every method; nothing is ever re-solved.

`8 calls`

2 Disagreement gate

Group the 4 CoT answers into distinct candidates. Fewer than 2 readable candidates means unanimous: keep the majority answer and stop.

`0 calls`

3 Contrastive verification

All distinct candidates in ONE call, one representative solution each, vote counts hidden. Per candidate: strategy, bindings, conditions, verdict.

`1 call`

4 Decision rule

Code, no model. Strictly SOUND = verdict SOUND and zero MISMATCH and zero VIOLATED. Exactly one wins; several, most votes; none, keep the initial answer.

`0 calls`

5 Scoring

Gold enters here and nowhere else. Paired fixes and breaks against CoT maj@4, exact McNemar tests.

`0 calls`

Dropped from the earlier six-stage design, with the evidence:

Recognition call (near chance: 52% catch, 45% false alarm — the free disagreement signal catches 86%) · **Three alternative strategies + recovery** (strategy validity had no effect on solve accuracy, 10.6% vs 10.3%, and the forced detour caused the breaks) · **Adjudication vote** (re-introduced the solver's own confidence)

Cost in practice: the gate retires 74.1% of instances at zero cost, so ESV adds 0.26 calls and 1,038 tokens per instance (+43.6%) rather than the 4,008 a verification call costs when it fires. Full accounting: the two compute slides after the results.

Inside the Verification Call

One call per disagreeing instance covers every candidate at once. The checker fills a fixed form of 4n+1 tags; code then converts that form into a pass or fail per candidate.

The form, per candidate

`strategy` the method in one sentence

`bindings` per given the method relies on:

```
"quote from the problem" -> how used
-> MATCHES or MISMATCH
```

`conditions` HOLDS or VIOLATED here

`divergence` first disagreeing step, ruled on from the problem text

`verdict` SOUND or FLAWED, failure type BINDING / METHOD / COMPUTATION, flawed step, corrected fact

Code then counts the literal tokens — it never reads the prose:

candidate	MATCH	MISM.	HOLDS	VIOL.	verdict	passes
35 (3 votes)	3	1	1	1	FLAWED	no
245 (1 vote)	4	0	2	0	SOUND	YES

The structured checks can overrule the verdict.
A candidate passes only if the verdict says SOUND and it logged zero MISMATCH and zero VIOLATED. A checker that writes SOUND after quoting a misused given is rejected as self-contradictory.

Why hide the vote?

The prompt states it outright: "they are listed in arbitrary order; how many attempts reached each answer is not shown and is not evidence." The vote is already computed for free at the gate, so a second opinion that merely reproduces it adds nothing. Universal Self-Consistency is the counter-example: shown all four responses with duplicates and asked for majority consensus, it chose correctly on 67.3% of disagreeing instances — only +3.2 pp over plain majority voting, and not significant ($p = 0.189$). ESV, with multiplicity removed, reaches 83.9%.

Worked Example: One Instance, Two Checkers

Pair 131, Gemma 3 4B, held-out. Candidates: 35 (3 votes, the trap) and 245 (1 vote, correct). Overturning a 3-vote majority is the hardest case in the set.

Self-check Gemma verifies Gemma

bindings for the trap candidate:

```
"...twice as far as Brian walks in one day."   MATCHES
all four givens marked MATCHES, zero MISMATCH
```

divergence ruling:

```
"The problem statement explicitly states that Sofie walks
twice as far as Brian walks in *one day*, so 7S = 2(7B)"
```

It quotes the right words, then writes the wrong equation.

verdicts: `35 -> SOUND` `245 -> FLAWED`

final answer 35 — wrong

Independent check gpt-oss-120B

bindings for the trap candidate:

```
"...twice as far as Brian walks in one day."
   -> used as 7S = 2(7B) -> MISMATCH
conditions: equation must represent the stated relation -
> VIOLATED
```

divergence ruling:

```
"The problem says Sofie's 7-day total equals twice
Brian's one-day distance ... thus Solution 2's step is
supported."
```

corrected_fact: `"7S = 2B"`

verdicts: `35 -> FLAWED (BINDING)` `245 -> SOUND`

final answer 245 — correct

Same prompt, same candidates, same hidden votes, same parser, same decision branch. Only the model filling in the MATCHES / MISMATCH column changed — and that column flips the answer.

Experimental Setup

Everything is pre-registered. The decision rule and the verifier were frozen on development pairs 0-59 before the held-out run; pairs 60-197 were never used for any tuning.

Go / no-go, fixed in advance:
ESV must beat CoT maj@4 by at least 3 pp AND beat Universal Self-Consistency. Otherwise the project becomes an analysis paper.

Why these solvers: larger models score about 97% on GSM8K, leaving a 2.1 pp ceiling — too little headroom to test any selection method. These three leave 6.1 pp.

1. **Dataset:** 198 frozen GSM8K pairs = 396 instances per solver; 390 after excluding 6 disputed labels
2. **Solvers:** Qwen 2.5 7B Instruct, Mistral Nemo 12B, Gemma 3 4B — via OpenRouter, T = 0.7, n = 4
3. **Verifier:** GPT-OSS-120B for the independent arm; the solver itself for the self-check arm
4. **Splits:** dev = pairs 0-59 (357 instances) · held-out = pairs 60-197 (813 instances)
5. **Baselines:** Direct QA maj@4, CoT pass@1, CoT maj@4, Universal Self-Consistency, CoT pass@4 oracle
6. **Scoring:** Deterministic normalization, then exact McNemar on paired fixes and breaks
7. **Blindness:** No stage sees gold. The checker never sees vote counts or the gold answer.

Results: Held-Out Pairs 60-197

method	Qwen 7B	Nemo 12B	Gemma 4B	POOLED	fixes / breaks	p
Direct QA maj@4	18.1%	9.6%	6.6%	11.4%	6 / 635	<1e-99
CoT pass@1	87.6%	78.9%	89.1%	85.2%	–	–
CoT maj@4 (baseline)	90.4%	85.2%	90.8%	88.8%	–	–
Universal Self-Consistency	89.7%	86.7%	92.6%	89.7%	14 / 7	0.189
ESV, self-check	88.2%	79.0%	88.2%	85.1%	12 / 42	5e-05
ESV, independent check	94.1%	93.7%	94.5%	94.1%	45 / 2	1.6e-11
CoT pass@4 (oracle ceiling)	94.5%	95.2%	95.6%	95.1%	–	–

+5.3 pp

over CoT maj@4: 45 fixes, 2 breaks, p = 1.6e-11

+4.4 pp

over Universal Self-Consistency: +40 / -4, p = 1.7e-08

-3.7 pp

the same protocol with the solver checking itself: 12 fixes, 42 breaks

Dev set (pairs 0-59, 357 instances), same ordering: maj@4 87.7 · USC 88.5 · ESV-self 85.2 · ESV-independent 92.2 (+4.5 pp, 16 fixes / 0 breaks). Both pre-registered conditions pass on dev and on held-out.

By arm (all 198 pairs, pooled): originals 90.6 -> 94.4 · strategy-breaking P-prime 79.1 -> 87.5 (25 fixes, 0 breaks) · strategy-preserving 93.6 -> 97.6. The breaking arm is where the perturbations bite and where ESV recovers most.

3 solvers x 271 held-out instances. Reproduce: analyze_esv3.py --benchmark gsm8k --models small --min-pair 60 [--verifier-model openai/gpt-oss-120b]

Compute Profile: What Each Method Costs

No method re-solves the problem: all four share the same 4 CoT samples and add at most one call, so the baseline is a constant and the extra call is the entire difference. serial = billed compute time; path = what a user waits for, since the 4 samples are issued concurrently.

method	accuracy	calls	solver tok	verifier tok	total tok	vs maj@4	serial	path
Direct QA maj@4	12.1%	4.00	1,095	–	1,095	-54.0%	2.4 s	1.0 s
CoT pass@1	85.1%	1.00	595	–	595	-75.0%	4.8 s	4.8 s
CoT maj@4 (baseline)	88.5%	4.00	2,379	–	2,379	–	19.2 s	6.5 s
Universal Self-Consistency	89.3%	4.26	2,799	–	2,799	+17.6%	19.4 s	6.7 s
ESV, self-check	85.1%	4.26	3,158	–	3,158	+32.7%	23.5 s	10.8 s
ESV, independent check	**93.5%**	**4.26**	**2,379**	**1,038**	**3,417**	**+43.6%**	**26.4 s**	**13.7 s**
CoT pass@4 (oracle ceiling)	94.5%	4.00	2,379	–	2,379	–	19.2 s	6.5 s

The disagreement gate is the efficiency mechanism

Stage 1 is code over answers already paid for: free, and it retires three-quarters of the work.

```
fires on     303/1,170  (25.9%)
when fired   4,008 tok,   27.8 s
amortized    1,038 tok,    7.2 s
ungated      +168%, not +43.6%
```

Gated out = the maj@4 answer untouched: 0 tokens, 0 seconds.

+5.0 pp for +43.6% tokens and +0.26 calls

Not the 2x–5x a verify-everything pipeline costs. And the right answer is already in the 4 samples we paid for 94.5% of the time — ESV is buying selection, not reasoning, which is why its cost is a fraction of the baseline rather than a multiple of it.

The self-check arm is strictly dominated: +779 tokens and +4.3 s to lose 3.3 pp.

Pooled over 3 small solvers, all 198 pairs = 1,170 instances. Read off the usage and latency stored on every call, so it reproduces offline: analyze_esv3.py --benchmark gsm8k --models small [--verifier-model openai/gpt-oss-120b], section 'Compute cost'.

Accuracy vs Compute: the Trade-off

net = fixes - breaks, so a method that trades a correct answer for a wrong one is charged for both. ESV costs 2.5x USC's token overhead and returns 5.9x the net fixes, so it is 2.4x more token-efficient per net fix than USC — the baseline it was pre-registered to beat.

method	Δ accuracy	fixes / breaks	net	extra tok	extra wait	tok per net fix
Direct QA maj@4	-76.3 pp	7 / 900	-893	-1,284	-5.5 s	no net gain
Universal Self-Consistency	+0.9 pp	19 / 9	+10	+420	+0.2 s	49,112
ESV, self-check	-3.3 pp	18 / 57	-39	+779	+4.3 s	no net gain
ESV, independent check	**+5.0 pp**	**61 / 2**	**+59**	**+1,038**	**+7.2 s**	**20,585**
CoT pass@4 (oracle ceiling)	+6.1 pp	71 / 0	+71	0	0 s	0 (needs gold)

The spend is adaptive — it tracks where the gain is

	gate	maj@4	ESV	extra tok
originals	20.3%	90.6%	94.4%	+33.2%
strategy-breaking P'	**45.1%**	**79.1%**	**87.5%**	**+72.7%**
preserving P'	17.5%	93.6%	97.6%	+28.0%

Disagreement is a free proxy for "the strategy may have broken". Per solver the cost of a fix barely moves: 18.4k–26.8k tokens.

The honest caveat: the verifier is a bigger model

The 1,038 check tokens are billed by gpt-oss-120B, the 2,379 sample tokens by a 4–12B solver. With k = the verifier's price per token over the solver's:

k=1 3,417 tok +43.6% | 20,585 per fix
k=2 4,455 tok +87.3% | 41,170 per fix
k=5 7,569 tok +218% | 102,925 per fix

Break-even against USC is k = 2.4. No price list is assumed — read off your k.

But the two are not interchangeable at any price: USC's total gain is capped at +10 net fixes against ESV's +59, so more USC cannot reach 93.5%. ESV buys accuracy USC cannot — k only sets its price.

Latency is ESV's real cost: the check is strictly sequential (it needs all 4 answers), so the wait goes 6.5 s -> 13.7 s overall and 34.3 s on the 25.9% of instances that fire. CoT pass@1 is the latency frontier: 85.1% for 595 tokens and 4.8 s.

Stage-Level Metrics and What the Data Removed

The original six stages, measured. Four negative results on JEEBench told us which stages carry the effect; v3 keeps three and drops three.

original stage	status in v3	held-out measurement
Recognition	replaced by the free gate	the gate flags 217/813 instances and catches 85.7% of majority errors at zero API cost
Strategy verification	KEPT — contrastive, per candidate	wrong candidates 98.8% FLAWED, 69.5% with a quoted MISMATCH; right candidates 88.4% SOUND; precision 97.7%
Diagnosis	folded into verification	failure types 62% BINDING, 27% COMPUTATION, 11% METHOD — binding-dominant, matching the benchmark design
Three alternative strategies	dropped	strategy validity had no effect on solve accuracy (10.6% vs 10.3%); the forced detour caused the breaks
Recovery (re-solve)	dropped	the needed answer already existed among the samples in all 45 of 45 recovered cases: selection, not generation
Adjudication vote	replaced by the frozen code rule	the one-SOUND branch now runs at 97.7% accuracy (45 fixes / 2 breaks)

The pivot in one sentence

Old ESV tried to generate a better answer. v3 recognises that a better answer usually already exists among the samples, and verifies its way to it.

Remaining headroom for a repair stage

45 held-out instances end in none_sound; a correct candidate exists in only 19, bounding any stage-4 repair at about +2.3 pp.

Limitations and Next Steps

Pending control 1: judge-select

The verifier is stronger than the solvers, so a reviewer will ask whether the ESV structure matters at all or only the bigger model. The control: same verifier, same candidates, asked only which answer is correct, with no bindings protocol. If ESV beats it the structure earns its keep; if they tie, the claim narrows to independent verification works.

Pending control 2: same-size cross-verifier

Separates independence from capability. Nemo checks Qwen's candidates and Qwen checks Nemo's. If a same-size independent checker still beats self-checking, the finding is about independence rather than strength — a deeper and more useful claim.

Scope

One benchmark (GSM8K), three small open-weight solvers, integer answers only. The large-solver arm has a 2.1 pp ceiling and is reported as a no-headroom control rather than a test.

Generalisation conditions

ESV needs givens that can be quoted from the problem text. That holds for word problems, MATH-Perturb and code I/O contracts, and should weaken where difficulty lies in unstated domain knowledge. A 50-instance pilot gives a cheap pre-test: check how sharply the verdict separates right from wrong candidates before committing to a full run.

Where this stands: both pre-registered conditions pass on data that no tuning touched, the failure mode of self-verification replicates, and the mechanism is pinned down quantitatively. The two controls above decide how much credit the protocol takes versus the checker — they are the next runs, not open questions about the result.

Next: Expanding Beyond GSM8K

MATH-500

Competition mathematics, 500 problems, with symbolic and multi-step answers instead of integers. MATH-Perturb already shows that hard perturbations invalidate the original solution, so the strategy-breaking twin has a published precedent here. The test: does ESV hold up when the answer is an expression and scoring is no longer exact match?

LiveCodeBench

Code generation with executable tests, and release-date tagging that controls for contamination. The givens become an I/O contract — signature, constraints and worked examples are the most quotable conditions ESV could ask for. New problem: candidates are programs, so selection needs behavioural equivalence rather than answer matching.

CruxEval-O

Predict a Python function's output for a given input: execution reasoning rather than generation. A strategy here is a trace, and a breaking perturbation changes control flow so the remembered trace no longer applies. Best property: gold comes from running the code, so pairs can be validated by execution instead of model consensus.

What stays fixed

The same protocol throughout: frozen original / perturbed pairs verified blind by two models, the same four CoT samples shared by every method, the frozen decision rule, and gold entering only at scoring. Each new benchmark gets a 50-instance pilot before a full run, so one that cannot support the premise is identified cheaply.

Why these three: they move ESV along two axes at once — from integer answers to symbolic ones, and from stated givens to executable contracts. GSM8K showed the mechanism works where the conditions can be quoted; these three test where it should hold, and CruxEval-O is where a reused strategy is cheapest to prove wrong.